

European Flight Delay Prediction

Table of Contents

<i>Proposal</i>	<i>2</i>
<i>General Information</i>	<i>2</i>
<i>1. Raw Data Storage</i>	<i>2</i>
Bucket Structure & Permissions	3
Raw Data Storage Reproducibility.....	4
<i>2. Processed Data Storage & File Formats</i>	<i>4</i>
<i>3. Database / Object Storage Decision – Parquet, Google Cloud Storage</i>	<i>5</i>
<i>4. Data Versioning</i>	<i>6</i>
<i>5. Data Access</i>	<i>6</i>
<i>6. Data Split / Validation Strategy</i>	<i>8</i>
Data Split / Validation Strategy Reproducibility - TBC.....	8
<i>7. Feature Description</i>	<i>9</i>
<i>8. Data Types and Formats.....</i>	<i>10</i>
<i>9. Reproducibility of Pre-Processing.....</i>	<i>12</i>
Profiling/Raw data Analysis.....	13
Identifying Groups/Statuses for Sampling	14
Analysis and Sample set creation Reproducibility	16

Proposal

I plan to use the European Flight Delays 2023-2025 dataset from Hugging Face to build a classification system that will predict whether a flight is likely to be delayed by at least 15 minutes based on the departure and destination airports, operator, aircraft type and date etc.

This dataset contains around 15 million per-flight delay estimates for European commercial flights from January 2023 to December 2025, derived from the Open Performance Data Initiative (OPDI) flight-list data published by Eurocontrol.

- <https://huggingface.co/datasets/345rf4gt56t4r3e3/flight-delays-europe-2023-2025>

```
@dataset{flight_delays_europe_2023_2025,  
  title = {Flight Delays Europe 2023-2025},  
  author = {Open-source community},  
  year = {2026},  
  url = {https://huggingface.co/datasets/345rf4gt56t4r3e3/flight-delays-europe-2023-2025}  
}
```

General Information

Author	Sharon Maye (Student Number 20122716)
Course	SETU AI & Machine Learning Certificate - Data Handling and Infrastructure Project, 2026 Academic Year
Project Status	In Development
GitHub Repo	https://github.com/sharonjmaye/european-flight-delay-prediction
Data File Format	Apache Parquet
Data Storage	Google Cloud Storage Bucket

1. Raw Data Storage

Goal - Clearly states where the raw data will live, including the storage system/location and why it is appropriate.

The raw dataset will be stored in a Google Cloud Storage bucket at the following location

- Bucket Cloud Storage URL – <https://console.cloud.google.com/storage/browser/european-flight-delay-prediction>
- Bucket Cloud Storage URI - gs://european-flight-delay-prediction

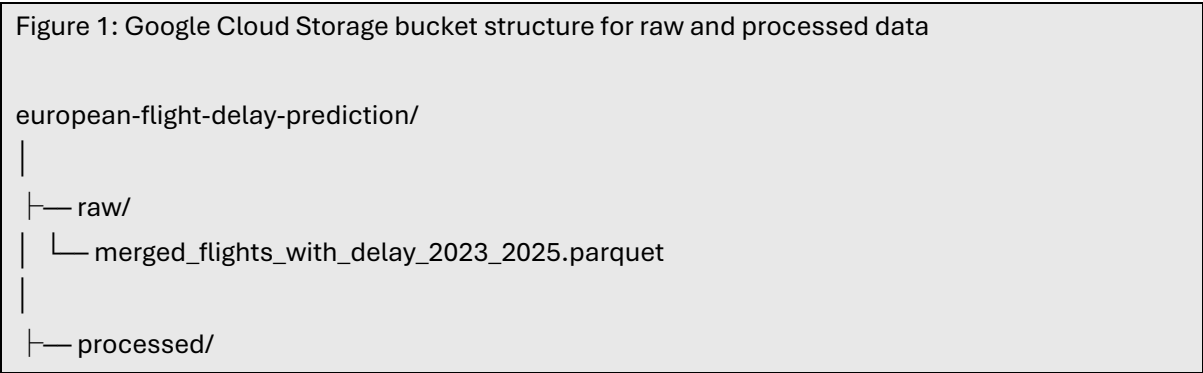
The raw dataset contains approximately 15 million flight records compressed into a single 787MB Apache Parquet file. The raw data will be retained this format because it is well suited to these type of large analytical datasets as it stores the data efficiently and allows selected columns to be read

without having to load the full dataset into memory. This will be particularly useful for initial analysis, sampling and feature selection

Google Cloud Storage is appropriate for the raw dataset because it is a large file based dataset rather than a transactional database and we want the raw data to be retained, accessed, downloaded and processed by notebooks and machine learning workflows rather than updated row by row. I also choose this storage option as it supports reproducibility and collaboration and can be easily accessed from Google Colab, Jupyter notebooks and other Google Cloud services if needed. This makes it easier for the project workflow to be reproduced for review and potential future reuse.

The raw dataset will be stored under a dedicated raw/ folder. Any sampling, cleaning, transformation, or feature engineering will be carried out separately and saved under the processed/ folder. This separation will ensure that the original raw dataset remains unchanged during the project and makes it possible to recreate the processed dataset from the original raw file if required.

Bucket Structure & Permissions



To support peer and lecturer review, the bucket has been configured with public read-only access. This allows reviewers to view and download the data, while preventing them from modifying or deleting any objects. This was done by granting the allUsers principal the following role - Cloud Storage → Storage Object Viewer

Permissions

[View by principals](#) [View by roles](#)

[Grant access](#) [Remove access](#)

Enter property name or value

☐	Type	Principal ↑	Name	Role	
☐	allUsers			Storage Object Viewer	Edit
☐	Editors of project: setu-data-handling			Storage Legacy Bucket Owner	Edit
				Storage Legacy Object Owner	
☐	Owners of project: setu-data-handling			Storage Legacy Bucket Owner	Edit
				Storage Legacy Object Owner	
☐	Viewers of project: setu-data-handling			Storage Legacy Bucket Reader	Edit
				Storage Legacy Object Reader	

Raw Data Storage Reproducibility

The download of the raw dataset and uploading to the GCS bucket can be reproduced using the related Jupyter notebook listed below which is maintained within the GitHub repo and designed to run in **Google Colab**

Note: As most users will only have read access to the existing bucket referenced not all steps in the notebooks will work as expected. If you wish to run all steps please update the following configuration variables in Step 2 to one of your Google Cloud projects instead

- PROJECT_ID
- BUCKET_NAME

Item	Details
Notebook	notebooks/01_upload_raw_data_to_gcs.ipynb
Purpose	Loads the raw Hugging Face flight delay dataset and uploads it to Google Cloud Storage in Parquet Format
Output	Public URL - https://storage.googleapis.com/european-flight-delay-prediction/raw/merged_flights_with_delay_2023_2025.parquet Cloud Storage URI - gs://european-flight-delay-prediction/raw/merged_flights_with_delay_2023_2025.parquet
Recommended Runtime	Google Colab
Requirements	• Access to GitHub repo • Google Colab • Google Cloud Authentication • Packages listed in requirements.txt if not using Google Colab

2. Processed Data Storage & File Formats

Goal - Clearly describes where processed data will be stored and specifies appropriate file formats for raw and/or processed data, with enough detail to understand the intended representation.

As the raw dataset is very large with approximately 15 million records a smaller representative sample of approximately 100,000 records will be created from the raw data for analysis and model development. Further detail on how this sample is selected is provided in section '9. Reproducibility of Pre-Processing '.

The processed dataset will be stored in the same Google Cloud Storage bucket as the raw data, but in a separate processed/ folder - gs://european-flight-delay-prediction/processed/flights_downsampled_100k.parquet

Storing the processed data separately ensures that the original raw source data remains unchanged. It also supports reproducibility as the processed data can be reproduced from the raw datasets using the associated notebook.

The processed data will also be stored in Apache Parquet format for similar reasons to the raw data. Parquet reduces file size and storage cost, preserves data types and allows for specified columns to be read without having to load the whole dataset into memory. This makes it suitable for exploratory analysis, feature engineering and machine learning model development

3. Database / Object Storage Decision – Parquet, Google Cloud Storage

Goal - Explicitly considers whether a database, object storage, file system, or other solution is needed. The choice is justified in relation to the project's data and access requirements.

For this project, I decided to use Google Cloud Storage rather than a relational database, data warehouse, data lake or local file system.

The original raw data is one large Parquet file, and the processed 100,000 record sample will also be stored as a Parquet file in a Google Cloud Storage bucket. This is appropriate because it is reliable and can be easily accessed by my notebooks in Google Colab.

A database would be useful if I needed to update records regularly, manage lots of related tables or run a lot of SQL queries but that should not be necessary for this project.

Local file system would make it harder to reproduce.

A data warehouse such as BigQuery could be useful later for large-scale SQL analysis, but it is not required for the current file-based notebook workflow.

The main workflow here will be loading the dataset, analysing it, creating a smaller sample and then using that sample for modelling.

By storing the files in Google Cloud Storage, the raw and processed datasets have a clear cloud location that can be accessed from Google Colab.

The raw data will be stored in a raw/ folder and the processed sample will be stored in a separate processed/ folder. This keeps the original data unchanged and makes it easier to understand which file is the source data and which file is the working dataset used for analysis and modelling.

Overall, Google Cloud Storage is the most suitable choice for this stage of the project because it gives me a simple, cloud-based place to store large Parquet files, access them from Colab, and keep the project reproducible.

4. Data Versioning

Goal - Explains how different versions of the data will be identified and tracked, including changes to datasets where relevant.

Data versioning will be managed using Google Cloud Storage Object Versioning on the project bucket.

Object versioning has been enabled on the GCS bucket which will mean if a file is accidentally overwritten or replaced the previous version can be recovered. This is particularly important for the raw dataset because it helps protect the original source from accidental changes

The bucket has been configured with the following lifecycle rules, providing a balance between protecting against accidental updates and avoiding unnecessary storage costs from old versions accumulating over time

- Maximum 5 versions per object
- Noncurrent versions expire after 7 days

Rules Add a rule Delete all				
Action	Object condition	Works with		
Delete object	Object is noncurrent 5+ newer versions	Object versioning		
Delete object	7+ days since object became noncurrent	Object versioning		

The raw folder will contain the original source dataset while the processed folder will contain datasets created from the raw data such as the 100,000 record sample used for analysis and modelling. If the processed dataset is regenerated the new file will replace the current object while GCS Object versioning will retain the most recent previous versions.

5. Data Access

Goal - Clearly explains how the system/code will access the data, including relevant paths, APIs, credentials/access mechanisms, or other interfaces as appropriate.

The project data will be accessed using notebooks running in **Google Colab**. The data is stored in a **Google Cloud Storage** bucket called european-flight-delay-prediction.

The notebooks access the data using Python libraries such as pandas, pyarrow, and the Google Cloud Storage client library.

Google Colab is used as the main notebook environment. Access to the Google Cloud project is authenticated in the notebooks using user authentication

```
from google.colab import auth  
auth.authenticate_user()
```

This allows the notebooks to connect to the Google Cloud project and access the GCS bucket. The bucket has also been configured with public read-only access so that reviewers can view or download the data, but cannot modify or delete objects in the bucket.

The raw data ingestion notebook '01_raw_data_ingestion.ipynb' uses the Google Cloud Storage client to upload the original Parquet file into the bucket and then verifies that the uploaded object exists.

The analysis and downsampling notebook '02_analysis_and_downsampling.ipynb' reads the raw Parquet file directly from its GCS path using `pandas.read_parquet`. It only loads the columns required for profiling, sampling, and validation, rather than loading unnecessary columns into memory. The notebook then creates a reproducible 100,000-record sample using a fixed random seed.

This approach provides a clear and reproducible data access method because the notebooks use defined cloud storage paths rather than local file paths on one machine. The raw data remains in the raw/ folder, while the processed modelling dataset is stored separately in the processed/ folder.

Dataset	Access Path	Access Method
Bucket	gs://european-flight-delay-prediction	Managed using the Google Cloud Storage client library
Raw Dataset	gs://european-flight-delay-prediction/raw/merged_flights_with_delay_2023_2025.parquet	Read from Google Colab using <code>`pandas.read_parquet`</code> / <code>`pyarrow`</code>
Processed Sample	gs://european-flight-delay-prediction/processed/flights_downsampled_100k.parquet	Created by the analysis and downsampling notebook and stored back in GCS

6. Data Split / Validation Strategy

Goal - Provides a clear and appropriate data-splitting strategy: either cross-validation or a train/dev/test split. The description specifies how the split/validation will be performed and avoids inappropriate data leakage.

The split will be carried out after the representative 100,000-record sample has been created from the raw dataset. As the sample will already include records across the different years and delay classes, a random split is appropriate for this stage of the project. A fixed random seed will be used so that the same split can be reproduced.

To avoid data leakage, fields that directly reveal the delay outcome, such as delay_min, will not be included as model input features before the train, development, and test split is used for modelling.

The proposed split is:

Dataset	Percentage	Purpose
Training	80%	Used to train the model
Development	10%	Used to tune model settings and compare different models
Test set	10%	Final evaluation on unseen data

Where possible, I will split the data in a way that keeps a similar balance of delayed and non delayed flights in each group. As the delayed flights make up a small percentage of the overall raw data, I will need to ensure that the training development and test sets have similar percentage of delayed flights.

Data Split / Validation Strategy Reproducibility - TBC

The download of the raw dataset and uploading to the GCS bucket will be reproduced using the related Jupyter notebook listed below which is maintained within the GitHub repo and designed to run in **Google Colab** – **this notebook is yet to be created but will add in future phases**

Item	Details
Notebook	notebooks/
Purpose	
Output	Public URL - Cloud Storage URI -
Recommended Runtime	Google Colab

Requirements	• Access to GitHub repo • Google Colab • Google Cloud Authentication • Packages listed in requirements.txt if not using Google Colab
--------------	---

7. Feature Description

Goal - Features used by the AI system are clearly identified and described. It is possible to understand what each feature represents and how it is obtained or constructed.

The model will use selected fields from the flight dataset as input features to predict whether a flight is likely to be delayed by at least 15 minutes. The target variable for the model is `delayed_15min`, while the features are the input values used to help make that prediction.

To avoid data leakage i.e. so the model only has information that it would likely have at the time it is asked to make a prediction, the target variable `delayed_15min` will only be used as the output label and not as an input feature. Other associated fields which can be used to derive `delayed_15min` value such as `delay_min`, `first_seen`, `last_seen`, `duration_min` will also be excluded from the input fields for this reason.

The final set of features will be confirmed during later phases. At this stage, the candidate features are based on fields that are likely to be available before or at the time a flight delay prediction would be made. Unless otherwise specified they are all taken directly from the source dataset

Candidate Feature	Description	Reason for Inclusion
Dof	Date of flight	Flight delays may vary by time of year, day of week or season may be used to create date-based features such as month, day of week or season
Source_year	Year of flight	Enables analysis of yearly patterns and changes
Source_month	Month of flight	Enabled analysis of monthly and seasonal delay patterns
Adep	Departure Airport	Some airports may have higher or lower delay patterns
Ades	Destination Airport	Destination airport may influence expected delay risk

Route	Flight route	Some routes may be more delay-prone than others
Model	Aircraft model	Different models may have different delay patterns
typecode	Aircraft type code	Standardized Aircraft type feature
Icao_aircraft_class	Aircraft Classification	Allows Aircrafts to be grouped by class rather than individual model
Expected_duration_min	Expected flight duration in minutes	Longer or shorter flights may have different delay patterns

8. Data Types and Formats

Goal - Specifies relevant data types and formats (e.g., numerical, categorical, text, image; integer, float, string; CSV, JSON, Parquet, etc.) sufficiently to understand how the data will be handled.

The raw and processed datasets will be stored in Apache Parquet format. Parquet is suitable for this project because the dataset is structured and tabular, with each row representing one flight record and each column representing a flight attribute, metadata field, or target-related value.

The dataset contains a mixture of data types, including:

- Integer fields, such as id, source_year, and delayed_15min
- Float fields, such as expected_duration_min and delay_min
- String fields, such as adep, ades, model, typecode, icao_operator, and route
- Timestamp fields, such as dof, first_seen, and last_seen

These data types will be handled differently during analysis and modelling. Numerical fields can be used directly or transformed where appropriate. Categorical string fields, such as airports, aircraft type, operator, and route, will need to be encoded before they can be used by most machine learning models. Timestamp fields may be used to create derived features such as day of week or seasonal patterns.

The main target variable is delayed_15min, which is an integer field with values of 0 or 1. A value of 1 means the flight was delayed by at least 15 minutes, while a value of 0 means it was not. This will be used as the output variable for the binary classification model.

The following table identifies the main fields currently expected to be used for identification, sampling, analysis, feature engineering, or modelling. Some fields are marked as candidate predictors because they may be useful for predicting whether a flight is delayed

Column	Data Type	Description	Intended Use
id	Integer	Unique record identifier	Record identification / sampling
dof	Timestamp	Date of flight	Candidate predictor / feature engineering
adep	String	Departure airport	Candidate predictor
ades	String	Destination airport	Candidate predictor
typecode	String	Aircraft type code	Candidate predictor
version	String	Dataset/source version information	Metadata / review
source_month	String	Source month associated with the flight record - YYYYMM	Sampling / temporal analysis
source_year	Integer	Source year associated with the flight record - YYYY	Sampling / temporal analysis
expected_duration_min	Float	Expected flight duration in minutes	Candidate predictor
delay_min	Float	Observed flight delay in minutes	Target-related – exclude from predictors
delayed_15min	Integer	Binary indicator identifying flights delayed by at least 15 minutes – 0/1	Target variable
route	String	Flight route derived from departure/destination	Candidate predictor

Currently the plan is to not include the following fields for modelling unless further analysis determines their relevancy

Column	Data Type	Description	Notes
--------	-----------	-------------	-------

icao24	String	ICAO 24-bit aircraft/transponder identifier	Review before modelling
icao_operator	String	ICAO aircraft/operator identifier	38.97% rows have missing data for this field
model	String	Aircraft model	30.3% rows have missing data for this field
icao_aircraft_class	String	ICAO aircraft classification	14.77% rows have missing data for this field
flt_id	String	Flight identifier	Review before modelling
adep_p	String	Additional departure airport field	Requires further investigation
ades_p	String	Additional destination airport field	Requires further investigation
registration	String	Aircraft registration	Review before modelling
first_seen	Timestamp	Timestamp when flight was first observed	Potential data leakage – investigate
last_seen	Timestamp	Timestamp when flight was last observed	Potential data leakage – investigate
unix_time	Float	Unix-based time value	Requires further investigation
duration_min	Float	Observed flight duration in minutes	Potential data leakage – investigate

This approach ensures that the relevant data types and formats are clearly identified before modelling begins. It also separates fields that are suitable for immediate analysis from fields that require further investigation, reducing the risk of using inappropriate variables in the model.

9. Reproducibility of Pre-Processing

Goal - Describes how the data collection process can be reproduced, including data sources, collection procedures, relevant parameters/settings, and/or code where appropriate.

As previously mentioned a decision was made to include a smaller representative dataset of 100,000 records for this project.

In order to determine which records to include in the sample dataset I firstly profiled the existing data, split the data into important groups/Strata and then took a sample from each group in the same proportion as it appears in the full dataset so that this sample dataset would keep the same overall balance as the full 15 million record raw dataset.

The result of this analysis was the following sampling approach

Sample Size	100,000 records
Method	Proportional stratified sampling
Stratification Fields	source_month and delayed_15min preserving both monthly/time distribution and delay/non delay balance
Random Seed	42

See the sections below which contain more detail on the initial analysis and strata identification in the sections below and the reproducibility section which has the notebook details that can be used to reproduce all of this.

Profiling/Raw data Analysis

The following is an overview of the profiling done on the raw data using the main columns/fields that we are interested in

Total Rows	15,300,770 - There are no duplicate or missing Id's
Departure Airports	991
Destination Airports	1075
Operators	915
Aircraft Models	5932
Routes	120,157
Year Distribution	• 2023 – 30.03 • 2024 – 33.50

	2025 – 36.36
Delayed 15 min	Note the class imbalance here, only 8.36% of flights are delayed by at least 15 minutes 0 – 91.64% 1 – 8.36%
Delay by Year	2023 - Delayed by at least 15 minutes – 6.97 - Not delayed by at least 15 minutes – 93.03 2024 - Delayed by at least 15 minutes – 7.43 - Not delayed by at least 15 minutes – 92.57 2025 - Delayed by at least 15 minutes – 10.37 - Not delayed by at least 15 minutes – 89.63
Missing Data	There is missing data for these columns – decided not to remove rows with this missing data because this would substantially change the population we are trying to represent for example if we dropped rows with missing icao_operator we would be removing almost 6 million flights - icao_operator – 38.97% - Model – 30.30% - icao_aircraft_class – 14.77% - Typecode – 3.87%

Identifying Groups/Statuses for Sampling

As we are trying to predict the chance of a flight being delayed by more than 15 minutes our target identifier is the delayed_15 min column.

As shown from the analysis above only 8.36% of flights in the raw dataset are delayed by at least 15 minutes. In order to ensure that our processed data set is representative of this raw data our 100,000-flight sample should contain approximately 8,360 delayed flights and 91,640 non-delayed flights.

Within the source dataset, the proportion of flights classified as delayed by at least 15 minutes varies over time, increasing by each year therefore, the sampling strategy should also preserve this temporal and delay-class distribution.

- 6.97% in 2023
- 7.43% in 2024
- 10.37% in 2025

Proposed sampling design - 100,000 records, proportionally stratified by source_month (YYYYMM) and delayed_15min, using random seed 42.

This results in a sample set of 100,000 records with the 72 strata/groups and the following distribution where the sample set distribution is very close to the raw data

Distribution	Raw Data	Sample Set
Year Distribution	• 2023 – 30.03 • 2024 – 33.50 • 2025 – 36.36	• 2023 – 30.03 • 2024 – 33.51 • 2025 – 36.46
Delayed 15 min	• 0 – 91.64% • 1 – 8.36%	• 0 – 91.64% • 1 – 8.36%

For more details on these Strata see SamplingAllocation.csv in the GitHub repo

The stratified sampling process identified the records to include in the 100,000-flight sample using their unique IDs. The complete records for those flights were then retrieved from the original raw Parquet dataset.

Any additional/unexpected class imbalance can be addressed later during modelling through appropriate algorithms, class weights, resampling of the training data only, threshold selection, and appropriate evaluation metrics.

Analysis and Sample set creation Reproducibility

The initial analysis, strata identification and sample set creation including uploading the sample set to the GCS bucket can be reproduced using the related Jupyter notebook listed below which is maintained within the GitHub repo and designed to run in **Google Colab**

Note: As most users will only have read access to the existing bucket referenced not all steps in the notebooks will work as expected. If you wish to run all steps please update the following configuration variables in Step 2 to one of your Google Cloud projects instead

- PROJECT_ID
- BUCKET_NAME

Item	Details
Notebook	notebooks/02_analysis_and_downsampling.ipynb
Purpose	Initial analysis, strata identification and creation and validation of sample set
Output	Public URL - https://storage.googleapis.com/european-flight-delay-prediction/processed/flights_downsampled_100k.parquet Cloud Storage URI - gs://european-flight-delay-prediction/processed/flights_downsampled_100k.parquet
Recommended Runtime	Google Colab
Requirements	• Access to GitHub repo • Google Colab • Google Cloud Authentication • Packages listed in requirements.txt if not using Google Colab